University of Pisa

Computer Engineering

I^2C Writing Master

Iacopo Massei

Academic Year 2025/2026

Contents

Chapter 1

Introduction

The I^2C is a serial communication protocol used to attach lower-speed peripheral ICs to processors and micro-controllers in short-distance and intra-board communication. As reported in Randall Hyde's book [1], before I^2C bus, different components of a computer system communicated with one another using 8 to 32 data lines and some number of address signals, requiring more space and increasing the amount of noise to deal with. In 1982 [2], Philips Semiconductors (now NXP Semiconductors) developed the I^2C protocol, alleviating these problems using just 2 wires: the data line (SDA) and the clock line (SCL).

1.1 Circuit description

The system is composed of one or more masters, that can write or read, and several slaves, all connected to the bus, with a bidirectional communication. Typical implementation is shown in Figure 1.1:

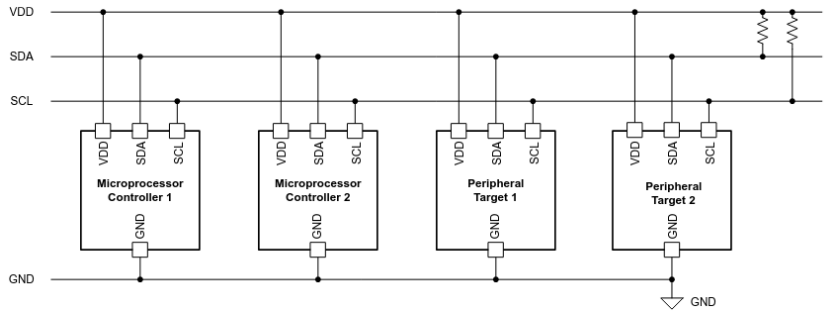

Figure 1.1: I^2C implementation [2].

1.2 Possible applications

The I^2C protocol is widely adopted in embedded systems, thanks to the low power consumption and space optimization, to interface microcontrollers with various peripheral ICs, such as sensors, execution elements and human interaction devices. One example is presented in [3], where the protocol is used to connect a master, the embedded computer, to all the devices installed for a smart home.

1.3 Writing Master

One possible implementation is the one reported in this documentation: the Writing Master. The device acts as the master in the I^2C protocol, but can only write data; no reading operation is implemented. The design is shown in Figure 1.2:

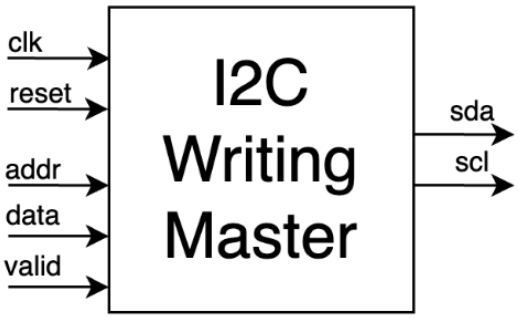

Figure 1.2: Writing Master design

The system is synchronized by a global clock (`clk`) and initialized by an active-low asynchronous `reset`. All the signals in input to the Writing Master come from a processor or a microcontroller to indicate when to communicate, using the `valid` signal, to whom, providing the 7-bit `addr` signal, and what, using the 8-bit `data` signal. After receiving the essential information, the master starts to communicate with the slave indicated in the `addr` signal using the `scl` and `sda` signals. The `sda` signal is used by the slave too during the acknowledgment phase. The clock used to communicate with the slave is 32 times slower than the system clock (`clk` signal). A possible use case of this implementation is with actuators or display drivers, for example, where the master sends a command and they simply answer with the ACK signal.

Chapter 2

Architecture

The architecture of the Writing Master is characterized by three main components:

- `REG_STATE LOGIC` (synchronous), for updating all the registers, i.e., current state, counters and data;
- `NEXT_STATE LOGIC` (combinatorial), to compute the next state;
- `OUTPUT LOGIC` (combinatorial), to generate the correct signals for the I^2C protocol and serialize data.

The following section present the block diagram to show how these components communicate and how the signals are used.

2.1 Block diagram

The Figure 2.1 shows how the three components communicate between them and which signals are used for the logic implementation:

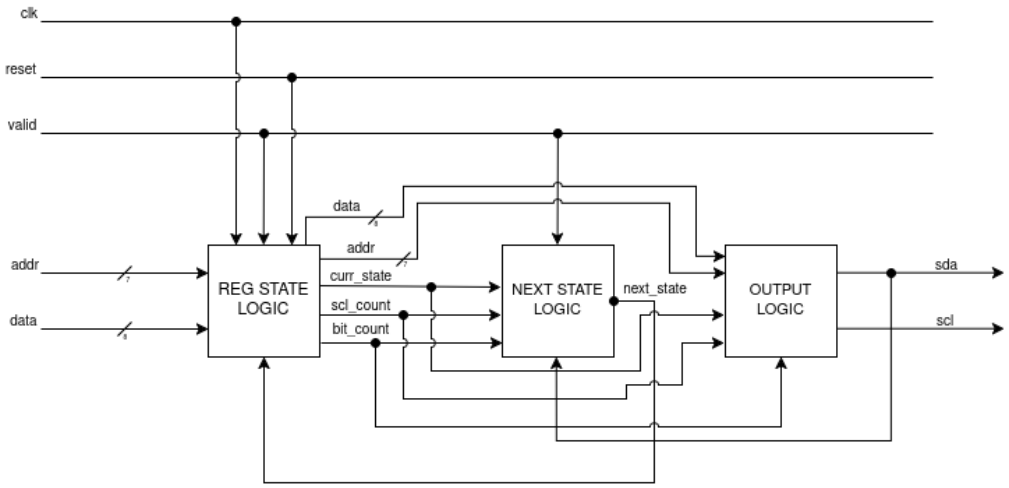

Figure 2.1: Writing Master block diagram

The `REG_STATE LOGIC` is the synchronous block of the architecture. It manages the registers and handles the reset phase (`reset` signal is 0). The `next_state` signal, coming from the `NEXT_STATE LOGIC`, drives the current state register, while the `addr` and `data` signals are sampled only when `valid` is 1. Regarding the counters, the `scl_count` signal is incremented by one at every clock cycle, whereas the `bit_count` signal increments every 32 clock cycles and only during transmission states. The `NEXT_STATE LOGIC` is one of the two combinatorial components and implements the logic to decide the next state where the system will be at the next rising edge of the clock. It needs all the counters to know when the last bit is transmitted and the I^2C cycle clock is over. Additionally, it uses the `valid` signal to start a transaction (consecutive or not) and checks `sda` signal to know if the acknowledgement signal is received.The last combinatorial component is the `OUTPUT LOGIC`, which sets the output signals to communicate using the I^2C protocol. Based on the `scl_count` signal, it generates a clock 32 times slower than the system clock, while the `bit_count` signal is used to select the correct bit to transmit.

Chapter 3

VHDL

In this chapter the VHDL description of the Writing Master is presented, starting from an external view, with the entity description, to the internal architecture of the device.

3.1 Entity declaration

Listing 3.1 reports the external description of the Writing Master, as presented in Figure 1.2:

```
entity writing_master is
     -- I/O ports definition
    port (
        clk : in std_logic;                         -- system clock signal
        reset : in std_logic;                       -- reset signal
        addr : in std_logic_vector(6 downto 0);     -- slave's address
        data : in std_logic_vector(7 downto 0);     -- writing data
        valid : in std_logic;                       -- input validity signal ('1' or '0')
        sda : inout std_logic;                      -- data line for serial communication
        scl : out std_logic                         -- device clock line
    );
end entity;
```

Listing 3.1: Writing Master Entity

The entity interfaces are presented by the `port` description, where all input and output signals are reported. As already said, the `sda` signal is a `inout` port because it is used both by the master and the slave. All the signals are based on the `std_logic` family, and not `bit` type, because they can assume other logic values rather than only `1` or `0`, as the `Z` logic value for the `inout` signal. Every signal is represented by one bit, except for `addr` and `data` signals that are `std_logic_vector` because they are parallel buses on 7 and 8 bits respectively.

3.2 Architecture description

This section provides the internal description of the entity, including the declarative part and the statement part of the architecture.

3.2.1 Declarative part

Listing 3.2 reports all the internal signals and types used to implement the logic of the Writing Master:

```
architecture structure of writing_master is

    type state_t is (IDLE, START, ADDR_STATE, ACK_ADDR, DATA_STATE, ACK_DATA, STOP);
    signal curr_state, next_state : state_t;
    signal scl_count :  unsigned(4 downto 0);               -- counter to set scl to '0'
     or '1' (32 times slower than clk)
    signal bit_count :  unsigned(3 downto 0);               -- index of the next bit to
     send (MSB first), max 9 (8 bit data + ACK)
    signal addr_signal :  std_logic_vector(6 downto 0);     -- set to addr when valid is
     '1'
    signal data_signal :  std_logic_vector(7 downto 0);     -- set to data when valid is
     '1'
    signal scl_signal : std_logic;                          -- used to read/set scl signal
     without error
    signal sda_signal : std_logic;                          -- used to read/set sda signal
     without error
```

Listing 3.2: Architecture declarative part

The type `state_t` defines all the states of the FSM. The `curr_state` and `next_state` signals are declared as `state_t` type to track the state. Regarding the counter signals, `scl_count` and `bit_count`, they are defined as `unsigned` type (5 and 4 bits respectively). Both signals are represented as a bus of wires, but they are interpreted as natural numbers allowing the use of basic arithmetic operations (`std_logic_vector` type is interpreted just as wires). Adding to this, the choice of the `unsigned` type, rather than `integer` type, is justified by the possibility to specify the exact number of bits needed (`integer` type occupies 32 bits) and to access the individual bits. The `addr_signal` and `data_signal` are used to register the values of the corresponding inputs when the `valid` signal is `1`. Finally, the `scl_signal` and `sda_signal` act as internal registers to update and hold the corresponding output signals.

3.2.2 Statement part

The statement part is divided into three processes corresponding to the three blocks illustrated in Figure 2.1:

- `p_STATE_REG`, corresponding to the `REG_STATE LOGIC` block;
- `p_NEXT_STATE_LOGIC`, corresponding to the `NEXT_STATE LOGIC` block;
- `p_OUTPUT_LOGIC`, corresponding to the `OUTPUT LOGIC` block.

These three processes are illustrated in detail in the following sections.

3.2.2.1 p_STATE_REG

```
1   p_STATE_REG: process(clk, reset)
2   begin
3       if reset = '0' then
4           -- initial state
5           curr_state <= IDLE;
6           scl_count <= (others => '0');
7           bit_count <= (others => '0');
8       elsif rising_edge(clk) then
9           -- device clock is 32 times slower than clock, except for IDLE state
10          curr_state <= next_state;
11          scl_count <= scl_count + 1;
12
13          case curr_state is
14              when IDLE =>
15                  scl_count <= (others => '0');
16                  bit_count <= (others => '0');
17
18                  if valid = '1' then
19                      addr_signal <= addr;
20                      data_signal <= data;
21                  end if;
22              when ADDR_STATE | DATA_STATE =>
23                  if scl_count = 31 then
24                      bit_count <= bit_count + 1;
25                  end if;
26              when ACK_ADDR =>
27                  bit_count <= (others => '0');
28              when ACK_DATA =>
29                  bit_count <= (others => '0');
30                  if (scl_count = 31) and (valid = '1') then
31                      addr_signal <= addr;
32                      data_signal <= data;
33                  end if;
34              when others => null;
35          end case;
36      end if;
37  end process;
```

Listing 3.3: p_STATE_REG process

The sensitivity list of this process is composed of two signals: `reset` and `clk` (the system clock). The process triggers whenever an event occurs on one of these two signals; the internal logic evaluates the reset phase and the rising edge of the clock. During the reset phase the `curr_state` is set to the `IDLE` state and all counters are set to `0`. On the rising edge of the clock the state register is updated by the `next_state` signal. Additionally, the internal counters and the data registers (`addr_signal` and `data_signal`) are updated accordin to the control signals.

3.2.2.2 p_NEXT_STATE_LOGIC

```
1   p_NEXT_STATE_LOGIC: process(curr_state, valid, scl_count, bit_count, sda)
2   begin
3       next_state <= curr_state;
4       case curr_state is
5           when IDLE =>
6               if valid = '1' then
7                   next_state <= START;
8               end if;
9           when START =>
10              if scl_count = 31 then
11                  next_state <= ADDR_STATE;
12              end if;
13          when ADDR_STATE =>
14              if (scl_count = 31) and (bit_count = 7) then
15                  next_state <= ACK_ADDR;
16              end if;
17          when ACK_ADDR =>
18              -- ACK signal is sda low
19              if scl_count = 31 then
20                  if sda = '0' then
21                      next_state <= DATA_STATE;
22                  else
23                      next_state <= STOP;
24                  end if;
25              end if;
26          when DATA_STATE =>
27              if (scl_count = 31) and (bit_count = 7) then
28                  next_state <= ACK_DATA;
29              end if;
30          when ACK_DATA =>
31              if scl_count = 31 then
32                  if sda = '0' then
33                      if valid = '1' then
34                          next_state <= START;
35                      else
36                          next_state <= STOP;
37                      end if;
38                  else
39                      next_state <= STOP;
40                  end if;
41              end if;
42          when STOP =>
43              if scl_count = 31 then
44                  next_state <= IDLE;
45              end if;
46      end case;
47  end process;
```

Listing 3.4: p_NEXT_STATE_LOGIC process

This process, corresponding to the `NEXT_STATE_LOGIC` block in Figure 2.1, implements the combinatorial logic to determine the next state based on the `valid` signal for the `START` state and on the `scl_count` signal to synchronize the clock of the I^2C protocol. Furthermore, to detect the end of the sending phase and to know when the acknowledgement signal is received, the internal logic evaluates the `bit_count` and `sda` signals, respectively. In the `ACK_DATA`

state, a check on the `valid` signal is performed to start a new, consecutive transmission without releasing the I^{2}C bus. To avoid inferred latches, a default assignment is provided at line 3 of Listing 3.4.

3.2.2.3 p_OUTPUT_LOGIC

```
1   p_OUTPUT_LOGIC: process(curr_state, scl_count, bit_count, addr_signal, data_signal)
2   begin
3       -- default ('0' with counter < 16, '1' >= 16)
4       scl_signal <= scl_count(4);
5       sda_signal <= '1';
6
7       case curr_state is
8           when IDLE =>
9               scl_signal <= '1';
10              sda_signal <= '1';
11          when START =>
12              scl_signal <= '1';
13              if scl_count < 16 then
14                  sda_signal <= '1';
15              else
16                  -- START signal with scl high and high to low transition of sda
17                  sda_signal <= '0';
18              end if;
19          when ADDR_STATE =>
20              if bit_count < 7 then
21                  -- MSB first order
22                  sda_signal <= addr_signal(6 - to_integer(bit_count));
23              else
24                  -- send write (W) command
25                  sda_signal <= '0';
26              end if;
27          when ACK_ADDR | ACK_DATA =>
28              sda_signal <= 'Z';
29          when DATA_STATE =>
30              sda_signal <= data_signal(7 - to_integer(bit_count));
31          when STOP =>
32              -- wait a little more to avoid a new START transition
33              if scl_count < 24 then
34                  sda_signal <= '0';
35              else
36                  -- STOP signal with scl high and low to high transition of sda
37                  sda_signal <= '1';
38              end if;
39      end case;
40  end process;
```

Listing 3.5: p_OUTPUT_LOGIC process

The `p_OUTPUT_LOGIC` process, corresponding to the `OUTPUT_LOGIC` block in Figure 2.1, sets the outputs of the I^{2}C protocol. It evaluates the `curr_state` signal, uses the `scl_count` signal to generate the I^{2}C clock and relies on the `bit_count` signal for the correct bit to transmit on the `sda` line. As in the previous process, a default assignment is provided at lines 4 and 5 of Listing 3.5 to avoid inferred latches.

Chapter 4

Testing implementation

The implementation described in the previous chapter is tested in four different scenarios:

- **one message**, the simplest one where the master has to send just one byte to an existing slave;
- **multiple messages**, after transmitting one message successfully, the master receives a new message to transmit to the same slave or to another, without releasing the I^2C bus;
- **NACK received**, where the master receives a NACK (`sda` signal high with `scl` high) in the `ACK_ADDR` and `ACK_DATA` states;
- **asynchronous reset**, where the `reset` signal is activated (`0`) during the protocol.

All tests are executed with different inputs. Specifically, the **multiple messages** test is performed by transmitting to the same slave or to a different one. Furthermore, the system clock period is set to 8 ns, which corresponds to a frequency of 125 MHz. This value was specifically chosen to match the Programmable Logic (PL) clock of the ZyBo development board, as presented in the course materials.

For the details of the test bench implementation, the *tb_writing_master.vhd* file is provided.

Bibliography

[1] R. Hyde, *The Book of I2C: A Guide for Adventurers*. No Starch Press, 2022.

[2] J. Wu, "A basic guide to i2c," *Texas Instruments*, p. 132, 2022.

[3] L. N. Pintilie, T. Pop, I. C. Gros, and A. M. Iuoras, "An i2c and ethernet based open-source solution for home automation in the iot context," in *2019 54th International Universities Power Engineering Conference (UPEC)*, IEEE, 2019, pp. 1–4.